Foundations of Neural Networks

Module 0: A Quick Refresher for the Rest of the Course

Cinvestav Unidad Guadalajara | Confidential / No distribution

Agenda

- Building blocks
- Loss, training, and backpropagation
- Regularization
- Evaluation

Objectives

Recap important concepts learned throughout the course, and refresh them

Cinvestav Unidad Guadalajara | Confidential / No distribution

Building Blocks

Cinvestav Unidad Guadalajara | Confidential / No distribution

A neuron: a weighted vote

A neural network is a stack of very simple decisions. A single **neuron** computes a weighted vote over its inputs, adds a bias, and passes the result through a nonlinearity.

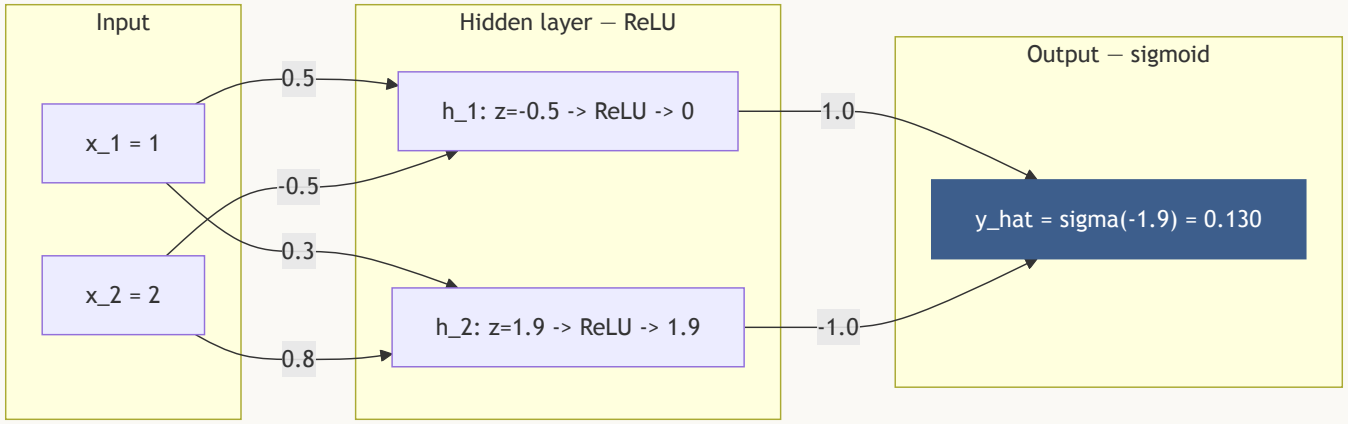
A whole **layer** of neurons does this at once, as a matrix multiplication — and stacking two layers just feeds one layer's output into the next:

$$h^{(1)} = g\big(W^{(1)}x + b^{(1)}\big)$$
$$h^{(2)} = g\big(W^{(2)}h^{(1)} + b^{(2)}\big) \tag{1}$$

Layer 1 multiplies the input x by its weights $W^{(1)}$, shifts it by $b^{(1)}$, and squashes it with g; layer 2 does the exact same thing to layer 1's output $h^{(1)}$.

Cinvestav Unidad Guadalajara | Confidential / No distribution

The idea in a picture



This network's weights are just numbers, chosen by training — a stack of `linear map + nonlinearity`. Without the nonlinearity, any depth collapses to a single linear map.

Cinvestav Unidad Guadalajara | Confidential / No distribution

Activation functions, briefly

A neuron's raw weighted sum is just a straight line — the **activation function** is what lets a network bend that line into something richer. It decides *how* a neuron "fires".

Name	Formula	Range	Notes
Sigmoid	$1/(1+e^{-z})$	$(0, 1)$	soft on/off switch; saturates at both ends
Tanh	$(e^{z}-e^{-z})/(e^{z}+e^{-z})$	$(-1, 1)$	same switch, centered at zero
ReLU	$\max(0, z)$	$[0, \infty)$	one-way valve — the modern default

Cinvestav Unidad Guadalajara | Confidential / No distribution

Inputs are vectors

Whatever the modality, the network sees a vector $x \in \mathbb{R}^d$: text becomes token vectors, audio becomes spectrogram frames, images become pixel values.

> Models don't read text or images — they read vectors.

Cinvestav Unidad Guadalajara | Confidential / No distribution

Regression and classification: the two common jobs

Nearly every task a network solves is one of these two:

	Regression	Classification
Predicts	a continuous number	which of K categories
Output layer	raw score, no activation	sigmoid ($K=2$) or softmax ($K>2$)
Examples	house price, temperature, a rating	spam vs. not spam, which digit, which language

> Same skeleton underneath — `linear map + nonlinearity` , stacked — only the **last** layer's shape and activation change to fit the job.

Cinvestav Unidad Guadalajara | Confidential / No distribution

How the network answers: softmax

Regression — the raw output *is* the answer. **Classification** — the network produces one score ("logit") per class, and **softmax** turns scores into a probability distribution: exponentiate every score (amplifying gaps), normalize so they sum to 1.

$$p_i = \text{softmax}(o)_i = \frac{e^{o_i}}{\sum_{j=1}^{K} e^{o_j}} \tag{2}$$

Cinvestav Unidad Guadalajara | Confidential / No distribution

Loss, Training, and Backpropagation

Cinvestav Unidad Guadalajara | Confidential / No distribution

The loss: a score of wrongness

The loss $\mathcal{L}$ is one number that says how far the prediction $\hat{y}$ is from the target y; training is nothing but "make this number smaller."

> The two standard losses aren't arbitrary recipes — MSE assumes Gaussian noise around a predicted value, cross-entropy assumes the model outputs class probabilities (log perform better i 0 to 1 values).

In general terms

For **any** layer, backprop reuses the same three moves:

1. Start from the output error — how wrong the prediction was (from the loss).
2. **Route it back** through that layer's weights, then **gate it** by how "open" that layer's activation was.
3. The gradient for a layer's weights = (the routed, gated error) × (whatever came *into* that layer).

> **One forward pass + one backward pass yields every gradient** — this reuse is exactly what PyTorch's `.backward()` automates.

Cinvestav Unidad Guadalajara | Confidential / No distribution

When training goes wrong: vanishing gradients

Backprop reuses the *same* step at every layer — for a deep network, that means multiplying **many factors together** on the way back:

- If each factor is **smaller than 1**, the product **vanishes** toward zero the deeper it travels — sigmoid's derivative alone (capped at 0.25) can shrink a 10-layer gradient by up to $0.25^{10} \approx 10^{-6}$.
- If each factor is **larger than 1**, the product **explodes** instead — the standard fix is to clip the gradient's norm before using it.

Regularization

Cinvestav Unidad Guadalajara | Confidential / No distribution

Memorizing vs. learning

A model with enough parameters can *memorize* the training set — like a student who memorizes past exams instead of learning the subject: perfect on the practice test, lost on the real one.

Overfitting = the gap between training performance and unseen-data performance. Regularization trades a bit of training-set fit for better generalization: keep the weights small (L2), don't let neurons form fragile conspiracies (dropout), stop studying before you start memorizing (early stopping).

Cinvestav Unidad Guadalajara | Confidential / No distribution

L2 regularization

Add a penalty on the squared weight norm to the loss:

$$\tilde{\mathcal{L}}(\theta) = \mathcal{L}(\theta) + \frac{\lambda}{2} \|\theta\|_2^2 \tag{3}$$

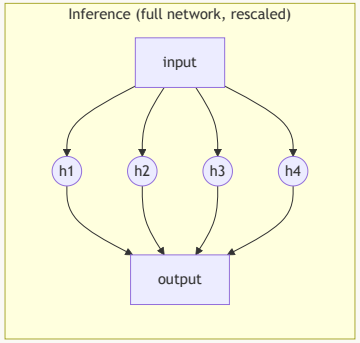
The update becomes $\theta_{t+1} \leftarrow (1 - \alpha\lambda)\,\theta_t - \alpha\nabla\mathcal{L}(\theta_t)$ — before every step, weights **shrink toward zero** by a constant factor. Hence **weight decay**: a weight survives at large magnitude only if the data keeps pushing it there. Smaller weights ⇒ smoother functions ⇒ lower variance, at the price of some bias.

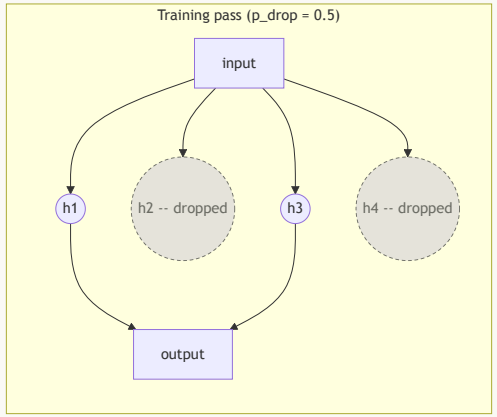
> Cousins: **L1/LASSO** pushes weights to *exactly* zero (sparsity).

Cinvestav Unidad Guadalajara | Confidential / No distribution

Dropout

During **training**, each hidden neuron is independently *dropped* (output set to 0) with probability p_{drop}; only surviving neurons forward signal and receive gradient updates.





Cinvestav Unidad Guadalajara | Confidential / No distribution

Dropout — rescaling and a practical bug

At inference nothing is dropped — the standard fix is **inverted dropout** (what PyTorch's `nn.Dropout` does): during training, divide surviving activations by $(1 - p_{\text{drop}})$, so inference needs **no change** at all.

> ⚠️ Dropout must be **ON** in training, **OFF** at evaluation — `model.train()` vs. `model.eval()`. Forgetting `eval()` is the most common dropout bug: predictions become randomly noisy.

Typical values: $p_{\text{drop}} \in [0.1, 0.5]$ for hidden layers; too high ⇒ *underfitting*.

Cinvestav Unidad Guadalajara | Confidential / No distribution

Evaluation

Cinvestav Unidad Guadalajara | Confidential / No distribution

You can't grade a student on what they studied from

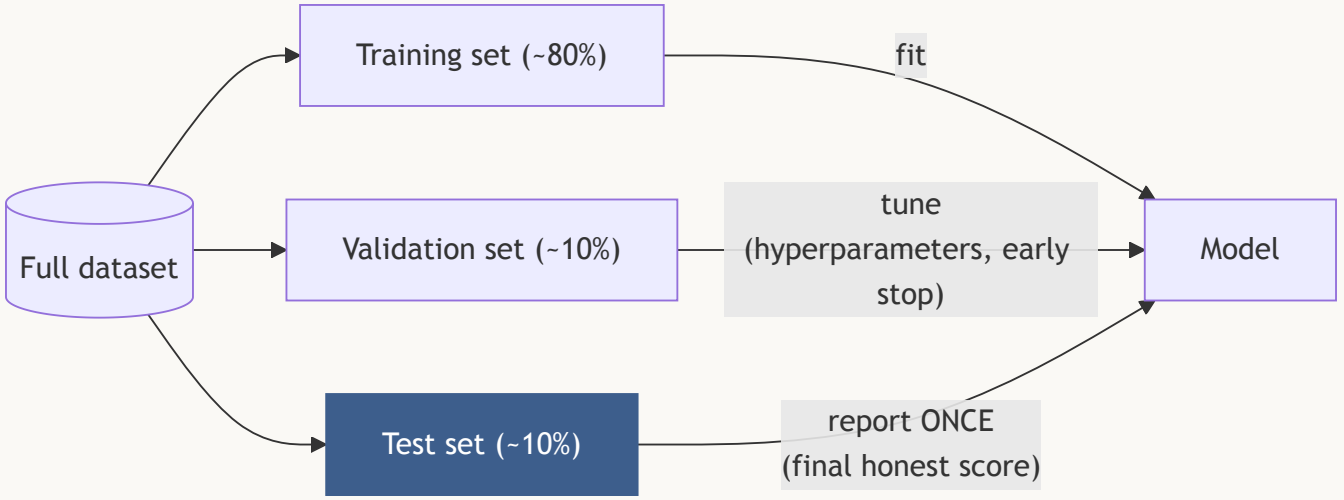
Split the data into three roles:

- **Training set** = the textbook — the model learns from it.
- **Validation set** = practice exams — used to choose hyperparameters.
- **Test set** = the final exam, sealed until the very end, used **once** to report honest performance.

> ⚠️ The moment you make *any* decision based on the test set, it silently becomes a validation set — and its grade is inflated.

Cinvestav Unidad Guadalajara | Confidential / No distribution

The idea in a picture



Typical proportions: **80/10/10** or **70/15/15**. Two classic traps: computing normalization statistics on **all** the data before splitting (leakage), and shuffling a time series so the model trains on the future.

Not all mistakes are equal

A single "percent correct" hides *which* mistakes the model makes — and mistakes are rarely symmetric (missing a disease ≠ a false alarm). The **confusion matrix** is the honest ledger behind every metric that follows.

Cinvestav Unidad Guadalajara | Confidential / No distribution

The confusion matrix

Every prediction on a positive/negative task falls into one of four boxes:

	predicted negative	**predicted positive**
actually negative	TN — correct	FP — false alarm
actually positive	FN — missed it	TP — correct

TP / TN = correct predictions of the positive / negative class. **FP** = predicted positive but actually negative (a *false alarm*, Type I error). **FN** = predicted negative but actually positive (a *miss*, Type II error).

Cinvestav Unidad Guadalajara | Confidential / No distribution

Made concrete: a spam filter

100 emails — 25 truly spam, 75 truly ham. The model flags 20 emails; 15 of them are truly spam.

	predicted ham	predicted spam
actually ham	TN = 70	FP = 5
actually spam	FN = 10	TP = 15

We'll compute every metric below on this same table, one at a time.

Accuracy

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \tag{4}$$

The fraction of **all** predictions that were correct. On the spam filter: $(15 + 70)/100 = 0.85$.

> ⚠️ **Accuracy fails under class imbalance.** If 95% of samples are negative, the useless "always predict negative" classifier scores 95% accuracy — with recall 0.

Cinvestav Unidad Guadalajara | Confidential / No distribution

Precision

$$\text{Precision} = \frac{TP}{TP + FP} \tag{5}$$

Of the emails the model **flagged** as spam, how many really are spam? On the filter: $15/20 = 0.75$ — 3 of every 4 flagged emails are truly spam.

Cinvestav Unidad Guadalajara | Confidential / No distribution

Recall

$$\text{Recall} = \frac{TP}{TP + FN} \tag{6}$$

Of the emails that **really are** spam, how many did the model catch? On the filter: $15/25 = 0.60$ — 40% of spam still slips through.

> Precision and recall **trade off**: flag more → recall ↑, precision ↓; flag less → the reverse. Neither one alone tells the whole story.

Cinvestav Unidad Guadalajara | Confidential / No distribution

F1 score

$$F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \tag{7}$$

The **harmonic mean** of precision and recall — not their average. It's dominated by whichever of the two is worse, so a great precision can't hide a terrible recall. On the filter: $F_1 \approx 0.667$.

Cinvestav Unidad Guadalajara | Confidential / No distribution

Bias-variance trade-off

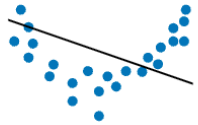
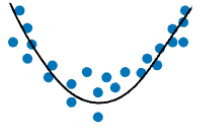
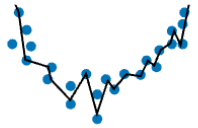
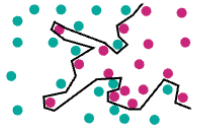
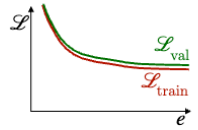
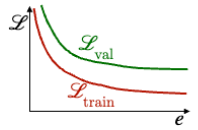
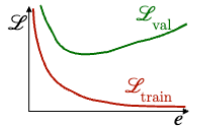
Bias = your aim is systematically off-center (the model family is too simple to represent the truth).

Variance = your throws scatter (retrain on a different sample and the model changes a lot).

> **Underfitting = high bias; overfitting = high variance.** You can't shrink both at once by turning the same knob — more model complexity trades bias away but buys variance, and vice versa. That's the trade-off: the sweet spot is somewhere in between, not at either extreme.

Cinvestav Unidad Guadalajara | Confidential / No distribution

Diagnosis table

	Underfitting	Just right	Overfitting
Symptoms	• High training error • Training error close to test error • High bias	Training error slightly lower than test error	• Low training error • Training error much lower than test error • High variance
Intuition			
Regression			
Classification			
Evolution of the loss			
Possible remedies	• Complexify model • Add more features • Train longer		• Regularize • Get more data

Cinvestav Unidad Guadalajara | Confidential / No distribution

What's next

Module 1 — Embeddings picks up exactly where this module's "inputs are vectors" remark left off: how does raw text become the vector $x \in \mathbb{R}^d$ this whole module assumed? Tokenization, then token embeddings, then document embeddings.

Cinvestav Unidad Guadalajara | Confidential / No distribution

References & further reading (1/2)

- Amidi, A. & Amidi, S. (2024), *Super Study Guide: Transformers and Large Language Models*, Part 1 "Foundations" — primary source for this module.
- Goodfellow, I., Bengio, Y. & Courville, A. (2016), *Deep Learning*, MIT Press — ch. 5 (bias–variance), ch. 6 (feedforward nets, MLE losses, backprop), ch. 7 (regularization).
- Prince, S. J. D. (2023), *Understanding Deep Learning*, MIT Press — ch. 3–9 (freely available online).
- Bishop, C. M. (2006), *Pattern Recognition and Machine Learning*, Springer — §3.2, bias–variance decomposition.

Cinvestav Unidad Guadalajara | Confidential / No distribution

References & further reading (2/2)

- Rumelhart, D., Hinton, G. & Williams, R. (1986), "Learning representations by back-propagating errors," *Nature* 323 — backpropagation.
- Srivastava, N. et al. (2014), "Dropout: A Simple Way to Prevent Neural Networks from Overfitting," *JMLR* 15.
- 3Blue1Brown, *Neural Networks* video series — recommended intuition companion.